

Building an Intraday Options Strategy ...and Everything That Went Wrong Along the Way

A teaching casebook on systematic trading: design, data, backtesting,
and the hard, honest lessons of quant research.

NOTE How to use this document

This is the real story of building one NSE intraday options strategy end-to-end. Each chapter pairs **what we did** with the **issues we hit** and the **lesson** learned. The mistakes are the most valuable part — most quant courses only show the polished result. Educational use only. Nothing here is financial advice.

1. The Goal — What We Set Out to Build

We wanted a **systematic intraday strategy** for Indian markets (NSE) that a retail trader could run from a laptop. The brief: scan a universe of liquid stocks and the two big indices (NIFTY, BANKNIFTY) all day, score each one objectively, and only flag a trade when strict, pre-defined conditions are met — removing emotion and guesswork. Orders are placed **manually** in the broker app; the software only generates and tracks signals.

The design principle from day one was **discipline over prediction**: we cannot forecast the market, but we can control which setups we take, how much we risk, and when we exit. The whole project became an exercise in honestly testing whether the chosen setups actually have an edge.

LESSON Define success before you build

We fixed a 'go-live bar' upfront: a strategy must show a win rate and profit factor above a set threshold across a minimum number of trades before any real money is risked. Without a pre-committed bar, you will always find a story to justify trading.

2. The Strategy — 3-Family Alpha Scoring

Every stock is judged by seven small checks, grouped into **three independent families**. Each family votes LONG, SHORT, or NEUTRAL. We group rather than count seven separate votes because several checks (momentum, trend strength, volume-break) all move together in a real trend — counting them separately would fake 'breadth'.

Family	What it measures	Weight
TREND	Momentum + trend quality + opening-range breakout	0.65
FLOW	Options positioning (PCR) + market regime (Nifty, VIX)	0.17
EVENT	News headlines + corporate filings (experimental)	0.18

Each family produces a **z-score** (how unusual today's reading is). We blend them into one number, the **alpha-z**, as a weighted average. The sign is the direction; the size is the conviction.

The two gates

A stock becomes a trade only if it clears BOTH gates — two independent methods agreeing:

- **Gate 1 — Alpha:** $|\text{alpha-z}|$ above 0.55 AND at least 2 of 3 families agree on direction.
- **Gate 2 — Confirmation:** a 5-minute opening-range breakout with a volume surge, same direction.

NOTE Why a 4th family was deleted

We originally had a mean-reversion family. A 30-day backtest showed it won only 47.6% of the time — no edge. We removed it. A factor that does not win has no place in the system, no matter how clever it sounds.

3. How the System Is Built

The software is organised into small, single-purpose modules so each piece can be tested alone:

Module	Job
config.py	All tunable parameters in one place
instruments.py	Map stock symbols to the broker's instrument keys
data_fetcher.py	Live prices + historical candles from the broker API
signals.py	The 3-family scoring and alpha-z maths
options.py	Resolve the exact option contract (strike, expiry, premium)
portfolio.py	Position sizing and instrument choice
agent.py	The orchestrator — scans every 5 minutes
trade_log.py	Records every signal and its outcome
ui_terminal.py	The dashboard you watch

The app runs **locally** and is fully autonomous: the Mac wakes itself at 8:55 AM, the app launches at 9:00, scans during market hours, and force-closes any position at 3:10 PM. A separate copy of the code lives on GitHub purely as a backup.

LESSON Separate the parts that can break

Keeping data, signals, sizing, and UI in separate modules meant that when something failed — and it did, repeatedly — we could isolate and fix one piece without touching the rest.

4. The Data Layer — Harder Than the Strategy

Most beginners assume the 'strategy' is the hard part. In reality, **getting clean, fast, correctly-identified data** consumed more effort than the trading logic. We switched the data source from Yahoo Finance to the broker's API (Upstox) specifically to remove latency.

ISSUE The API spoke a different language than we expected

Our first attempts to fetch candles returned 404 and 400 errors. The broker had moved to a new API version (V3) with a completely different URL structure, AND it required **ISIN-based instrument keys** (e.g. NSE_EQ|INE467B01029) instead of human symbols (NSE_EQ|TCS). Three mismatches at once — endpoint, key format, and response parsing — all had to be fixed before a single price came through.

LESSON Read the current API docs, don't assume

We solved it by reading the live API documentation, then auto-downloading the broker's full instrument master (96,000+ instruments) and building a symbol-to-key lookup that refreshes weekly. Never assume the API behaves the way a tutorial from last year says it does.

ISSUE Real-time streaming was not available

We wanted WebSocket tick streaming for zero latency. The read-only data token returns HTTP 410 — that feature needs a paid trading token. We could not have it.

LESSON Engineer within your real constraints

Instead of streaming, we made REST polling fast: fetch all prices in ONE batched call (0.2s for 100 names instead of 18s one-by-one), cache daily history so it is fetched once per day, and run the scan on 12 parallel threads. Result: a full 97-instrument scan in a few seconds — fast enough that prices don't drift.

5. The Bug That Made the Strategy Invisible

This is the single most important lesson in the entire project. When we first ran a proper backtest to answer 'how many signals does this generate per day?', the answer was **zero — every day, forever**. The strategy looked alive on the dashboard but would never have fired a single trade.

The cause was one line of maths. To turn a raw reading into a z-score, the code did:

```
momentum_z = stats.zscore([momentum_value])
```

A z-score measures how far a value sits from the average of a **distribution**. But we passed it a **single number**. The z-score of one number is mathematically undefined — the function returned nan ('not a number') every time. That nan poisoned the whole calculation, so the alpha-z was always nan, so no stock ever passed Gate 1.

ISSUE A silent failure that looked like success

Nothing crashed. No error appeared. The dashboard scanned, the lights blinked, everything 'worked' — it just silently produced no trades. Had we trusted the polished UI, we'd have paper-traded a dead system for a month and concluded 'the market gave no signals'.

LESSON Test the OUTPUT, not the appearance

We only caught it because we asked a blunt quantitative question ('how many signals in 30 days?') and got a suspicious answer (zero). The fix was to z-score each factor against its own ROLLING HISTORY (a real distribution), not a single point. Always verify a system produces the outputs you expect — a clean interface proves nothing.

6. The Win-Rate Illusion

Once signals fired, an early backtest showed a tempting **45% win rate**... that LOST money. Later we found a setting that showed **80%+** — also a trap. Understanding why is the heart of trading maths.

Win rate alone is meaningless without the **reward-to-risk ratio**. You can manufacture almost any win rate by choosing how far apart the target and stop sit:

Setup	Win rate	Reality
Tiny target, huge stop	80–90%	Each rare loss erases ~10 wins — loses money
Huge target, tiny stop	20–30%	Wins are big but rare — can still make money
Balanced 2:1	~33% breakeven	Needs a genuine edge to profit

NOTE The breakeven formula every student must know

With a reward-to-risk of R:1, you break even at a win rate of $1 / (1 + R)$. At 2:1, that's 33%. So a '34% win rate' at 2:1 is NOT skill — it is exactly what random entries produce. A high win rate with a bad reward-to-risk is the most common way beginners fool themselves.

LESSON Judge a system by EXPECTANCY, not win rate

Expectancy = (Win% × average win) – (Loss% × average loss). A 45% system with 2.5:1 reward beats a 60% system with 1:1. We learned to always report win rate, reward-to-risk, AND expectancy together.

7. Why We Moved to Buying Options

Testing on the underlying stock showed almost no directional edge at a sane 2:1 reward-to-risk — about a 30% win rate, which is the random baseline. A 5% intraday move in a stock is rare, so targets were hard to reach. We switched to **buying options** (CALL for bullish, PUT for bearish — never selling).

The key insight: with an option you exit on the **option premium**, not the stock price. Because of leverage, a small move in the stock can swing the premium 10%+, so a modest, achievable target becomes meaningful. Indices (NIFTY, BANKNIFTY) were added because their options need little capital (~Rs 12,000 and ~Rs 28,000 per lot).

ISSUE We could only backtest options on recent data

Historical option backtesting needs the premium history of the exact contract that was at-the-money on each past day. Expired contracts are removed from the free instrument master, and the broker's expired-data API requires a paid plan (our token got a 401). So we could only test the most recent ~30 days of option premium — a real, unavoidable limit on how far back the options study could go.

Tuning the option trade — every lever tested

We then methodically swept every parameter on the recent option data:

Lever	What we tested	Best choice
Exit	Premium target/stop combos	+10% target / -20% stop
Strike	ITM-2 to OTM+2	OTM+1 (one strike out)
Cutoff	1:00 / 1:30 / 2:00 / 3:00 PM	1:00 PM
Time window	9:45 AM to 1:00 PM buckets	12:30–1:00 PM strongest

NOTE A counter-intuitive finding on stops

A WIDER stop gave a HIGHER win rate. Option premiums are noisy; a tight 5% stop gets hit by random wiggles before the +10% target is reached. The 20% stop let trades breathe — 88% win in one sample vs 50% with a 5% stop. The 'safe-sounding' tight stop was actually worse.

8. The Overfitting Trap

At one point the request was: 'take all the winning trades and design signals that match them, to get a 70%+ win rate.' This is the single most dangerous idea in quant trading, and worth dwelling on.

ISSUE Fitting rules to past winners is cheating yourself

If you look at which trades won and then write rules that select exactly those, you can show ANY win rate on history — 90%, even 100%. But you have fitted to noise, not signal. Live, it collapses, because the future's winners are different from the past's.

LESSON Always validate out-of-sample

We split the data: find the pattern on the first 60% of days (the 'learn' set), then test it on the last 40% it had never seen (the 'prove' set). If a high win rate survives on unseen data, it might be real. If it collapses, it was overfitting. This train/test discipline is non-negotiable.

NOTE The student who chose wisely

Late in the project, a backtest suggested a narrow 12:30–1:00 PM window hit 77% vs 72% for the wider window. The right call was made: KEEP the wider window, because the 5-point gain rested on only 13 trades — well inside the noise band. Resisting a tempting-but-fragile number is exactly the discipline this whole project teaches.

9. The Sample-Size Problem That Never Went Away

Across dozens of backtests, one issue dominated every conclusion: **too few trades**. A selective intraday strategy with a 1 PM cutoff produces only about 14 signals a month, clustered on ~8 of 22 trading days. Most days are blank.

This means every shiny number — 72%, 77%, 88% — rested on 13 to 34 trades. At those sizes, a couple of different outcomes flips the win rate by 10+ points. We watched the 'same' setup read 77% on 13 trades and 50% on 14 trades from an overlapping window. Both were 'true'; both were noise.

Backtest	Trades	Reported win rate	Trustworthy?
20-day options	13	77%	No — too few
30-day options	14	50–64%	No — too few
120-day underlying	379	~30% at 2:1	Yes — but no edge

LESSON A number from 15 trades is a guess, not a result

Statistical confidence needs samples — ideally 100+ trades. Below that, treat every win rate as a hypothesis. The honest response to a thin sample is not to pick the prettiest number, but to say 'we don't know yet' and go collect more data.

10. Costs — The Silent Edge-Killer

Several setups showed a small positive 'gross' expectancy of about +0.06% per trade. Then we modelled real costs: brokerage, STT, exchange fees, GST, and stamp duty total roughly 0.10% round-trip for intraday equity.

The maths that ends many strategies

Gross edge per trade: +0.06% | Round-trip cost: –0.10% gives **Net: –0.04% — a loss.**

To get a high win rate you needed a tiny target (0.2–0.3%), but a tiny target is smaller than the cost of trading. The edge existed on paper and was eaten entirely by friction. This is why options, with their larger percentage moves, were the only place the maths could even potentially work.

LESSON Always subtract costs BEFORE celebrating

A backtest that ignores brokerage and taxes is a fantasy. The first thing to do with any positive result is subtract realistic round-trip costs. Many 'profitable' retail strategies are break-even or losing once friction is honest.

11. The Honest Verdict

After testing every lever — strike, target, stop, cutoff, entry-time, instrument — on real Upstox data, the conclusion was disciplined and unglamorous:

- The entry signals show **no proven directional edge** at a sane reward-to-risk.
- High win rates appeared only on samples too small to trust, or vanished after costs.
- The buy-option version (OTM+1, +10%/–20%, 1 PM) looked promising (~72%) but on just 18 trades.

- Nothing here is yet a proven money-maker — and we wrote that plainly into the dashboard itself.

NOTE The only honest next step

No backtest can settle it, because of the sample-size and cost limits. The real test is FORWARD: run the system live in paper mode, let it record 30+ real option outcomes with real fills and costs over the coming weeks, and only then judge whether the edge is real. Forward evidence cannot be curve-fitted.

LESSON The most professional outcome is sometimes 'not proven'

It would have been easy to tune numbers until a backtest sparkled and call it a winner. Refusing to do that — and saying clearly 'the edge is unproven, here is exactly why' — is the real mark of quantitative maturity. Protecting capital from a false positive is worth more than any backtest.

12. Ten Lessons to Take Away

#	Lesson	Why it matters
1	Version-control everything from day one.	The original code was lost because it lived untracked inside another project's folder. We rebuilt it and gave it its own git repository plus an off-site backup. Never trust a single copy.
2	A clean dashboard proves nothing.	The biggest bug produced zero trades while looking perfectly alive. Test outputs, not appearances.
3	Win rate without reward-to-risk is meaningless.	Judge by expectancy. Know that $1/(1+R)$ is your breakeven win rate.
4	A wide stop can beat a tight stop.	In noisy instruments, tight stops get shaken out. Test, don't assume.
5	Never fit rules to past winners.	Always validate on out-of-sample data you have not looked at.
6	Small samples lie.	Below ~100 trades, a win rate is a hypothesis, not a result.
7	Subtract costs before celebrating.	Brokerage and taxes kill thin edges. Model them first.
8	Know your data's limits.	We could not backtest expired options or stream ticks — and we said so, rather than pretending.
9	Engineer within real constraints.	Batching, caching, and parallelism gave near-real-time speed on a read-only token.
10	'Not proven' is a valid, professional answer.	The forward paper-test is the only honest judge. Protect capital first.

Institutional Trader — Strategy Build Casebook | For classroom and educational use only. Not investment advice. Markets carry risk of loss.